



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

14 мая 2025 г.

Цель работы

Шейдеры OpenGL

Целью данной лабораторной работы является переработка лабораторной работы №6 с использованием шейдеров на языке GLSL в среде OpenGL. Это позволяет повысить производительность и гибкость визуализации 3D-сцены, а также реализовать настраиваемое освещение и текстурирование.

Теория

Шейдеры — это маленькие программы, которые выполняются на GPU и управляют визуализацией объектов. Современный OpenGL (начиная с версии 3.3 Core Profile) требует использование собственных шейдеров вместо фиксированного функционала. Основными типами шейдеров являются:

- **Вершинный шейдер (Vertex Shader)** — отвечает за обработку каждой вершины: трансформации, нормали, координаты текстур и пр.
- **Фрагментный шейдер (Fragment Shader)** — отвечает за цвет фрагментов (пикселей), вычисляя освещение, цвет материала, текстуры и т.д.

Кроме того, используется набор uniform-переменных, передаваемых из CPU на GPU, для задания таких параметров, как матрицы трансформации (model, view, projection), параметры источников света и свойств материалов.

Типичный pipeline с шейдерами включает следующие этапы:

1. Генерация и буферизация данных о вершинах (позиции, нормали, текстурные координаты).
2. Компиляция и линковка шейдеров.
3. Установка uniform-переменных.
4. Отрисовка объектов с использованием VAO/VBO и шейдерной программы.

Реализация

Для реализации лабораторной работы был использован модуль `shaders.py`, включающий вершинный и фрагментный шейдеры, а также функции создания VAO и шейдерной программы.

Вершинный шейдер

```
1 #version 330 core
2 layout(location = 0) in vec3 aPos;
3 layout(location = 1) in vec3 aNormal;
4 layout(location = 2) in vec2 aTexCoord;
5
6 out vec3 FragPos;
7 out vec3 Normal;
8 out vec2 TexCoord;
9
10 uniform mat4 model;
11 uniform mat4 view;
12 uniform mat4 projection;
13
14 void main()
15 {
16     FragPos = vec3(model * vec4(aPos, 1.0));
17     Normal = mat3(transpose(inverse(model))) * aNormal;
18     TexCoord = aTexCoord;
19     gl_Position = projection * view * vec4(FragPos, 1.0);
20 }
```

Объяснение:

- `aPos`, `aNormal`, `aTexCoord` — входные данные на каждую вершину: позиция, нормаль, текстура.
- `FragPos`, `Normal`, `TexCoord` — передаются во фрагментный шейдер.
- Преобразования выполняются с помощью переданных `uniform`-матриц `model`, `view`, `projection`.

Фрагментный шейдер

```

1 #version 330 core
2 in vec3 FragPos;
3 in vec3 Normal;
4 in vec2 TexCoord;
5
6 out vec4 FragColor;
7
8 uniform sampler2D texture1;
9
10 uniform vec3 lightPos;
11 uniform vec3 viewPos;
12 uniform vec3 globalAmbient;
13 uniform vec3 lightColor;
14 uniform vec4 matAmbient;
15 uniform vec4 matDiffuse;
16 uniform vec4 matSpecular;
17 uniform float shininess;
18 uniform bool useTexture;
19
20 void main()
21 {
22     vec3 objectColor;
23     if (useTexture) {
24         objectColor = texture(texture1, TexCoord).rgb;
25     } else {
26         objectColor = vec3(1.0, 0.5, 0.31);
27     }
28
29     vec3 norm = normalize(Normal);
30     vec3 lightDir = normalize(lightPos - FragPos);
31     vec3 viewDir = normalize(viewPos - FragPos);
32     vec3 reflectDir = reflect(-lightDir, norm);
33
34     vec3 ambient = globalAmbient * matAmbient.rgb;
35     float diff = max(dot(norm, lightDir), 0.0);
36     vec3 diffuse = diff * matDiffuse.rgb * lightColor;
37     float spec = pow(max(dot(viewDir, reflectDir), 0.0), shininess);
38     vec3 specular = spec * matSpecular.rgb * lightColor;
39
40     vec3 result = (ambient + diffuse + specular) * objectColor;
41     FragColor = vec4(result, 1.0);
42 }

```

Объяснение:

- Рассчитываются компоненты освещения: ambient, diffuse, specular.

- Если активирована текстура, берется цвет из текстуры; иначе используется базовый цвет.
- Все компоненты суммируются и формируют итоговый цвет пикселя.

Создание VAO и загрузка данных торуса

```

1 vao = glGenVertexArrays(1)
2 glBindVertexArray(vao)
3
4 # Объединяем вершины, нормали и текстурные координаты
5 vertex_data = np.hstack([torus.vertices, torus.normals, torus.texcoords
6     ])
7 glBindBuffer(GL_ARRAY_BUFFER, vertex_data.nbytes, vertex_data,
8     GL_STATIC_DRAW)
9
10 # Индексы торуса (EBO)
11 glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, torus.indices.nbytes, torus.
12     indices, GL_STATIC_DRAW)

```

Объяснение:

- Используются VAO/VBO/EBO для загрузки данных на GPU.
- Устанавливаются указатели на вершины, нормали и текстурные координаты.
- Это необходимо для последующей отрисовки через glDrawElements.

Компиляция и связывание шейдеров

```

1 vs = compile_shader(VERTEX_SHADER_SRC, GL_VERTEX_SHADER)
2 fs = compile_shader(FRAGMENT_SHADER_SRC, GL_FRAGMENT_SHADER)
3 program = glCreateProgram()
4 glAttachShader(program, vs)
5 glAttachShader(program, fs)
6 glLinkProgram(program)

```

Объяснение:

- Компилируются вершинный и фрагментный шейдеры.
- Они связываются в единую шейдерную программу, которая затем используется при отрисовке.

Результаты

После интеграции шейдеров приложение демонстрирует корректную работу с динамическим освещением, текстурированием и реалистичным отражением. Основной объект сцены (тор) отображается с плавными бликами и цветами, отражающими параметры материала.

Реализация позволила отказаться от устаревшего фиксированного конвейера и перейти к современному способу управления рендерингом с использованием программируемых блоков.

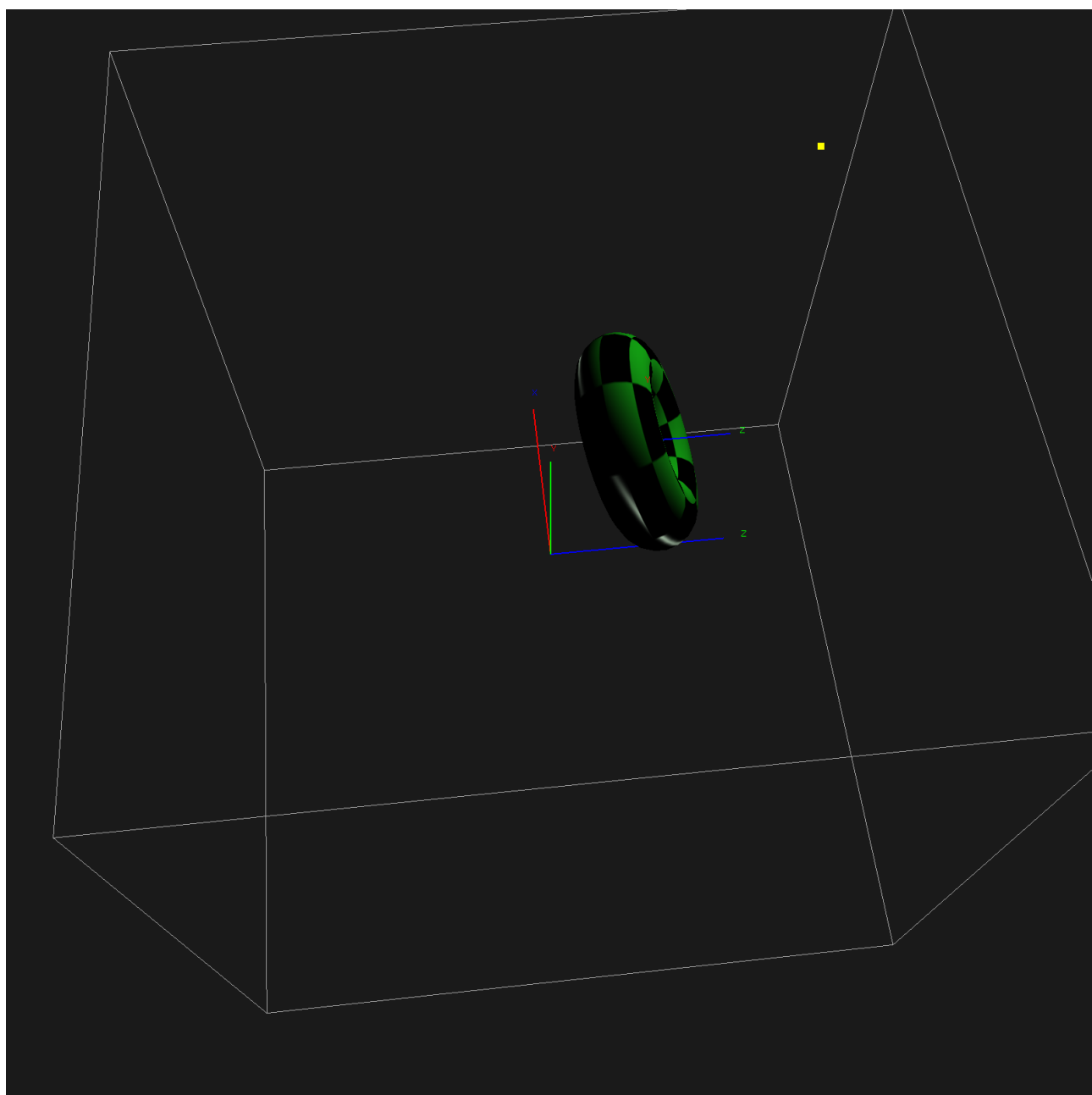


Рис. 1 — Торус в граничном кубе

Вывод

В рамках лабораторной работы была реализована переработка лаборатории №6 с применением вершинных и фрагментных шейдеров. Это позволило значительно улучшить качество визуализации и гибкость управления параметрами освещения и материалов. Программируемый конвейер OpenGL требует больше начальных усилий по настройке, но открывает широкие возможности для оптимизации и реалистичной графики. Данная работа дала практический опыт в компиляции и подключении GLSL-шейдеров, работе с буферами и организации данных на GPU.